

LBIST Adaptive T-period 알고리즘

생성형 AI 활용 성능 최적화 및 RTL 구현 프로젝트 보고서

이 문서는 LBIST(Logic Built-In Self-Test) Adaptive T-period 프로젝트를, 생성형 AI(Claude / Claude Code)를 활용해 알고리즘 성능을 최적화하고 RTL(Verilog)로 구현·검증한 확장 프로젝트의 전체 과정을 정리한 것이다. 기존 Python 재현 프로젝트와의 비교를 통해 AI 활용이 만들어낸 실질적 개선폭을 정량적으로 제시한다.

0. 프로젝트 개요

0.1 배경 및 동기

기존에 진행하던 LBIST Adaptive T-period 알고리즘은 '현재 MISR 신호 활동성(Hamming Distance) 추이를 관찰해 향후 필요한 테스트 주기(T-period) 조정을 예측하는 적응형 제어' 구조를 갖는다. 기존 프로젝트는 이 예측 로직을 논문에서 제시한 고정 휴리스틱 파라미터(threshold 계수 α , 조정 폭 ΔT 등)로 구현했는데, 이 파라미터들이 성능에 미치는 영향과 최적값 여부는 별도로 검증되지 않은 상태였다. 이 한계를 보완하기 위해 생성형 AI를 활용해 ①파라미터를 체계적으로 자동 탐색·최적화하고 ②최적화된 알고리즘을 실제 하드웨어(RTL)로 구현·검증하는 확장 프로젝트를 진행했다.

0.2 프로젝트 범위

- 1단계 — Python 알고리즘 성능 최적화: Optuna 베이지안 최적화(TPE)로 하이퍼파라미터 자동 탐색
- 2단계 — RTL 구현: 최적화된 알고리즘을 Verilog로 구현, Claude Code 에이전틱 루프(코드 생성→시뮬레이션→디버깅 반복)로 완성
- 3단계 — 검증 심화: 실제 게이트 레벨 회로(CUT)를 합성해 결합 주입 검증까지 확장, AI 주도 커버리지 클로저 실습 수행

0.3 사용 도구

- Claude Code (VS Code 확장) — 에이전틱 코드 생성, 시뮬레이션 실행, 디버깅
- Optuna 4.9.0 — 베이지안 최적화(TPE Sampler), fANOVA 파라미터 중요도 분석
- Icarus Verilog 12.0 (iverilog/vvp) — RTL 시뮬레이션
- Python 3.14 — 기존 LBIST 시뮬레이션 프레임워크(PRPG/MISR/결합 시뮬레이터)

1. 구현 과정 (이론)

1.1 기존 알고리즘의 이론적 구조

LBIST의 Adaptive T-period 제어는 다음 파이프라인으로 동작한다: PRPG(Pseudo-Random Pattern Generator, 16-bit Galois LFSR + phase shifter)가 매 사이클 스캔 패턴을 생성하고, 스캔 그룹 컨트롤러(GSS 방식)가 현재 활성 그룹에만 새 패턴을 적용하며, 결합 시뮬레이터가 정상/결함 회로의 출력을 비교해 누적 결합 커버리지를 갱신하고, MISR(Multiple-Input Signature Register)이 회로 출력을 16-bit 시그니처로 압축한다. 이 시그니처 간 Hamming Distance(HD)의 변화 추이를 Adaptive T-period 컨트롤러가 관찰하여 테스트 주기 T 를 조정한다.

컨트롤러의 핵심 로직: $\text{threshold} = \alpha \times \text{baseline}$; if $\text{avg_hd}(\text{sliding window, 크기 } W) < \text{threshold}$: $\text{consecutive_low} += 1$; if $\text{consecutive_low} \geq K$: $T = \min(T + \Delta T, T_{\text{max}})$ # '정체(stall)' 판정 → 주기 증가; else: $\text{consecutive_low} = 0$

baseline은 초기 calibration 구간(calibration_patterns = T_init × calib_multiplier) 동안 관찰된 HD의 평균으로 설정된다. 즉 전체 알고리즘은 6개의 하이퍼파라미터(T_init, T_max, delta_T, K, W, alpha, calib_multiplier)로 정의되며, 기존 프로젝트는 이 값들을 논문에서 제시한 고정값(T_init=128, delta_T=128, alpha=0.95, K=3, W=16, calib_multiplier=1.5)으로 그대로 사용했다.

1.2 AI 활용 방법론 (1단계 — 파라미터 최적화)

1.2.1 문제 설정

6개 하이퍼파라미터가 이루는 탐색 공간에서, ISCAS'89 벤치마크 회로 4종(s1494, s5378, s9234, s13207)에 대한 평균 결함 커버리지를 최대화하는 조합을 찾는 것이 목표다. 이는 각 파라미터 조합을 평가하는 데 실제 결함 시뮬레이션(수십 ~ 수백 초)이 필요한 블랙박스 최적화 문제이며, 그리드 서치나 순수 랜덤 서치로는 비효율적이다.

1.2.2 Optuna 베이지안 최적화(TPE)

Optuna의 TPE(Tree-structured Parzen Estimator) Sampler를 사용했다. TPE는 지금까지 시도한 파라미터와 결과의 분포를 학습해, '좋은 결과를 낼 확률이 높은 영역'과 '낮은 영역'을 구분하는 두 개의 확률 밀도 함수를 추정하고, 그 비율이 높은 지점을 다음 후보로 제안하는 방식이다. 초반 n_startup_trials(10회)는 순수 무작위 탐색으로 초기 분포를 확보하고, 이후 20회는 이 대리모델 기반 제안으로 진행되도록 설계했다.

목적함수: score = avg_final_coverage(4개 회로 평균, %) - 0.01 × avg_speed_frac. avg_speed_frac = 각 회로의 patterns_to_reach(target) / num_patterns 평균 (0 ~ 1, 작을수록 빠름). target은 각 회로의 Fixed-T(256) 최종 커버리지.

계수 0.01은 매우 작게 설정하여 커버리지(0 ~ 100 스케일)가 압도적 우선순위를 가지되, 커버리지가 사실상 동등할 때만 수렴 속도가 순위를 가르도록 했다. T_max(1024)는 논문 스펙 상수로 취급해 탐색 대상에서 제외했다.

1.2.3 RTL 변환 이론

최적화된 알고리즘을 하드웨어로 옮기는 과정에서 두 가지 이론적 문제를 다뤘다.

(1) 부동소수점 연산의 고정소수점 근사: 하드웨어는 float 곱셈($\alpha \times \text{baseline}$)을 직접 지원하지 않으므로, α 를 Q24 고정소수점 정수($\text{ALPHA_FIXED} = \text{round}(\alpha \times 2^{24})$)로 근사하고, 비교 부등식 ' $\text{avg_hd} < \alpha \times \text{baseline}$ '을 정수 교차곱 형태로 재구성했다: $\text{avg_hd} < \alpha \times \text{baseline} \Leftrightarrow \text{window_sum} / \text{window_count} < \text{ALPHA_FIXED} \times \text{baseline_sum} / (\text{CALIB_PATTERNS} \times 2^{24}) \Leftrightarrow \text{window_sum} \times \text{CALIB_PATTERNS} \times 2^{24} < \text{ALPHA_FIXED} \times \text{baseline_sum} \times \text{window_count}$

이렇게 하면 나눗셈 없이 정수 곱셈·비교만으로 원래의 실수 비교를 재현할 수 있으며, baseline/window 자체는 정수 합(sum) 형태로 유지되어 정밀도 손실이 전혀 없다. α 의 근사 오차만 존재하며, 이는 실제 값과 약 3×10^{-8} 수준으로 HD가 정수라는 점을 감안하면 실질적으로 원본과 동일한 판정 결과를 보장한다.

(2) 동기 회로의 신호 전파 순서: PRPG → MISR → HD 계산 → 컨트롤러로 이어지는 한 사이클 내 데이터 흐름을, Verilog의 논블로킹 대입(Non-Blocking Assignment, NBA) 스케줄링 규칙에 맞게 설계해야 한다. 같은 클럭 엣지에서 갱신되는 두 레지스터 중 하나가 다른 하나의 '갱신 후 값'을 그 사이클에 조합적으로 참조하려면, 레지스터의 현재값이 아니라 다음 상태(next_state)를 노출하는 포트가 별도로 필요하다.

1.3 Claude Code 에이전틱 워크플로우

RTL 구현은 사람이 스펙과 검증 기준을 제시하고, Claude Code가 (1) 기존 Python 코드 정독 및 알고리즘 요약 → (2) 골든 레퍼런스 벡터 생성 스크립트 작성(Python 원본 클래스를 직접 import해 정답 시퀀스 생성) → (3) RTL 모듈 작성 → (4) self-checking testbench 작성 → (5) iverilog 컴파일·시뮬레이션 실행 → (6) 실패 시 원인 분석 후 수정 → (5)로 재귀, 의 반복 루프를 수행하는 방식으로 진행했다. 사람은 각 단계의 실행을 승인하고, 설계 결정 지점(탐색 파라미터 범위, tie-break 규칙 등)에서 판단을 내렸다.

2. 어려움

2.1 최적화 실행 시간의 비현실적 소요

첫 번째 trial 실행 시간을 측정한 결과 약 117초가 소요되었고, 이를 기준으로 나머지 29 trial을 순차 실행하면 총 약 57분이 소요될 것으로 예상되었다. Claude Code 도구 자체의 단일 명령 실행 제한(약 10분)을 초과하는 규모였다.

2.2 부동소수점 → 고정소수점 변환의 정밀도 리스크

alpha와 같은 실수 파라미터를 정수로 변환할 때, 근사 오차가 threshold 비교의 판정 결과 자체를 뒤집을 위험이 있었다. 특히 avg_hd와 threshold의 값이 근접한 경계 상황에서 오차가 누적되면 RTL과 Python 모델의 동작이 갈릴 수 있었다.

2.3 테스트벤치 결함으로 인한 '거짓 통과(false pass)'

RTL 모듈 초기 구현 단계에서, 테스트벤치가 클록의 posedge 직후 곧바로 제어 신호(enable/update)를 변경하면서 DUT의 always 블록과 같은 시점에서 경쟁(race condition)이 발생했다. 그 결과 의도한 것보다 한 사이클 더 진행된 상태가 되어, PRPG 출력이 Python 레퍼런스와 전부 불일치하는 형태로 나타났다(300/300 사이클 불일치).

더 심각한 문제는 lbist_top 통합 테스트에서 발생했다. 이 테스트가 PRPG 패턴 폭(WIDTH)을 64(16의 배수)로 설정했는데, MISR의 phase-shifter 오프셋 공식이 16주기 구조를 가져, 이 폭에서는 매 채널이 정확히 4번씩 XOR-fold되어 시그니처가 항상 0으로 상쇄되는 특수 상황이 발생했다. 시그니처가 항상 0이었기 때문에 RTL과 Python이 '우연히' 계속 일치하는 것으로 나타나, 실제로는 존재하던 심각한 타이밍 버그가 통과 판정 뒤에 완전히 가려져 있었다.

2.4 PRPG-MISR 간 실제 RTL 타이밍 버그

위 우연한 통과 뒤에 숨어 있던 실제 결함은, PRPG의 상태 레지스터와 MISR의 시그니처 레지스터가 같은 클록 엣지에서 동시에 갱신될 때 발생했다. MISR이 '이번 엣지에 새로 생성된 패턴'이 아니라 '한 사이클 전 패턴'을 참조하고 있었다. 이는 시뮬레이터의 문제가 아니라, 조합적으로 파생된 신호가 같은 클록 엣지 내에서 아직 반영되지 않은 값으로 취급되는 Verilog NBA 스케줄링의 근본적 특성 때문이었으며, 실제 동기 회로 설계에서도 동일하게 발생할 수 있는 제약이었다.

2.5 게이트 레벨 실제 CUT 부재

초기 통합 RTL(lbist_top.v)은 실제 회로(CUT: Circuit Under Test) 없이 MISR의 입력을 PRPG 출력으로 직접 루프백하는 구조였다. 결함 시뮬레이터(fault_sim.py)가 Python 전용으로 구현되어 있어, 합성 가능한 게이트 레벨 CUT 모델이 프로젝트 내에 존재하지 않았기 때문이다. 이 상태로는 PRPG→MISR→컨트롤러로 이어지는 제어/데이터패스 통합은 검증되지만, 실제 결함이 회로를 통과해 MISR 시그니처에 반영되고 검출되는 전체 흐름은 검증되지 않는 한계가 있었다.

2.6 EDA 도구 지원 범위의 제약

Icarus Verilog은 SystemVerilog의 functional coverage 구문(covergroup/coverpoint)을 지원하지 않아(-g2012 옵션으로도 syntax error 발생), 표준 커버리지 정의 방식을 그대로 사용할 수 없었다.

3. 극복

3.1 백그라운드 프로세스 분리를 통한 실행 시간 문제 해결

단일 명령 실행 제한을 우회하기 위해, 나머지 trial 실행을 별도 백그라운드 프로세스로 분리하고 표준출력/에러를 로그 파일로 리다이렉트한 뒤, 주기적으로 로그와 진행 상황(trial_log.md의 항목 수)을 확인하는 방식으로 전환했다. Optuna의 study를 SQLite에 저장해 실행이 여러 세션에 걸쳐 나뉘어도 trial 번호가 이어지도록 설계했다. 실행 결과 총 30 trial이 약 2,665.9초(29 trial 기준, trial당 평균 91.9초)에 완료되었다.

3.2 Q24 고정소수점 변환 및 오차 검증

alpha를 24비트 소수부(Q24)로 양자화한 결과(ALPHA_FIXED = 14,111,141)의 복원값은 0.8410895466...로, 원본 alpha(0.8410895754890528)와의 오차는 약 3×10^{-8} 수준임을 사전에 계산해 확인한 뒤 RTL에 반영했다. baseline_sum, window_sum 등은 실수 평균이 아닌 정수 누적합 형태로 유지해 근사 오차가 개입할 여지를 원천적으로 차단했다.

3.3 비퇴화(non-degenerate) 테스트 조건으로 재검증

우연한 통과를 방지하기 위해, lbist_top의 검증 폭을 16의 배수가 아닌 63으로 변경하고 골든 레퍼런스를 재생성했다. 이 조건에서는 시그니처가 더 이상 항상 0으로 상쇄되지 않아 실제 의미 있는 값(예: 최종 시그니처 0x7c3e)으로 검증이 이루어지도록 만들었다.

3.4 next_pattern 포트 추가로 타이밍 버그 수정

PRPG 모듈에 현재 상태가 아닌 '다음 상태(next_state)'를 조합적으로 노출하는 next_pattern 포트를 추가하여, 같은 클록 엣지 내에서 MISR이 최신 패턴을 즉시 소비할 수 있도록 수정했다. 최상위 모듈의 관측용 pattern/response 신호도 레지스터화하여 post-edge 샘플링 시점과 어긋나지 않도록 정렬했다. 수정 후 비퇴화 조건(WIDTH=63)에서 기존 4개 테스트 스위트(PRPG/MISR/컨트롤러/통합 top)가 모두 정상적으로 재통과함을 확인했다.

3.5 게이트 레벨 CUT 합성을 통한 결함 검출 검증 완성

가장 작은 벤치마크 회로인 s1494(647 게이트, 14 입력, 25 출력)를 대상으로 .bench 넷리스트를 합성 가능한 gate-level Verilog(AND/OR/NOT 기반)로 변환하는 스크립트를 작성했다. 이 실제 CUT을 MISR 입력에 연결한 새 통합 모듈(lbist_top_with_cut.v, 기존 lbist_top.v는 보존)을 구성하고, force 방식으로 실제 stuck-at 결함을 하나 주입해 결함 유무에 따른 MISR 시그니처 변화와 검출 여부를 확인하는 self-checking testbench를 작성했다.

3.6 AI 주도 커버리지 클로저(Coverage Closure) 실습

SystemVerilog covergroup을 사용할 수 없어, 카운터/플래그 기반의 수동 커버리지 트래킹 방식으로 전환했다. MISR 모듈에 대해 21개의 커버포인트(response 전부-0/전부-1, 내부 상태 lsb 0/1, fold 연산의 16개 채널 각각, enable 도중 리셋 발생 시나리오)를 정의하고, 기존 무작위 자극만으로 얼마나 충족되는지 확인한 뒤, 미충족 항목을 채우는 지향성(directed) 자극을 스스로 설계해 추가하는 반복 절차를 수행했다.

4. 최종 결과

4.1 이론적 성과

- 파라미터 중요도(fANOVA) 분석 결과, 정제 판정 임계값 계수 α 가 성능을 좌우하는 압도적 1위 변수(중요도 0.7693)로 나타났으며, 나머지 파라미터(T_{init} 0.0723, ΔT 0.0475, W 0.0446, $calib_multiplier$ 0.0420, K 0.0244)의 영향은 상대적으로 작았다.
- 최적해는 기존 논문값 대비 T_{init} 을 크게 낮추고(128→33), ΔT 를 더 느슨하게(α 0.95→0.841), 관찰 윈도우를 더 넓게(W 16→34) 설정하는 방향으로 수렴했다. 이는 '초기부터 짧은 주기로 세밀하게 시작하되, 판정 기준은 느슨하게, 관찰 구간은 길게 가져가는' 전략이 이 알고리즘 구조에서 더 효과적임을 시사한다.
- RTL 구현 과정에서, 우연한 조건(패턴 폭이 16의 배수)이 심각한 타이밍 결함을 검증 결과상으로 완전히 은폐할 수 있다는 것을 실증적으로 확인했다 — 이는 검증 설계 자체의 타당성을 별도로 검증해야 한다는 방법론적 시사점을 준다.

4.2 수치적 성과

지표	값
Baseline(논문 기본값) 평균 결함 커버리지	62.71 %
최적화 후 평균 결함 커버리지 (Trial #22)	87.31 %
커버리지 개선폭	+24.60 %p
평균 수렴 속도(speed_frac, 낮을수록 빠름)	1.00 → 0.30 (약 3.3배 개선)
탐색 규모	30 trial (TPE, n_startup_trials=10)
최적 파라미터	$T_{init}=33$, $\Delta T=193$, $\alpha=0.8411$, $K=3$, $W=34$, $calib_x=2.183$
RTL 모듈 검증 결과	PRPG 300cyc · MISR 300cyc · 컨트롤러 600cyc · 통합 top 600cyc 전부 Python과 bit-exact 일치
게이트 레벨 CUT 결함 주입 검증	s1494(647 게이트) 실제 stuck-at 결함, RTL · Python 검출 결과 일치
기능 커버리지 클로저	21개 커버포인트 100% 달성 (2회 반복)

표 1. 최종 산출물 핵심 수치 요약

4.3 기존 프로젝트 대비 종합 비교

항목	기존 프로젝트 (Python 단독 재현)	이번 프로젝트 (AI 최적화 + RTL 구현)
파라미터 결정 방식	논문 값을 그대로 수동 적용	Optuna(TPE) 베이지안 최적화 자동 탐색
결함 커버리지 개선폭	+7 %p (논문 재현 기준)	+24.60 %p (baseline 대비, 4개 회로 평균)
파라미터 영향 분석	없음	fANOVA 기반 정량적 중요도 분석
검증 수준	Python 시뮬레이션	Python 시뮬레이션 + RTL(Verilog) bit-exact 검증
하드웨어 구현	없음	PRPG · MISR · Adaptive Controller · Top module 전체 구현

결함 검증 대상	추상화된 결함 모델(fault_sim.py)	실제 게이트 레벨 넷리스트(s1494) 합성 후 결함 실주입
검증 완결성 점검	—	기능 커버리지 21개 항목 100% 달성

표 2. 기존 프로젝트와 이번 확장 프로젝트의 비교

4.4 한계 및 향후 과제

- 게이트 레벨 CUT 검증은 벤치마크 4종 중 가장 작은 s1494 1개 회로에 한정되었다 — 나머지 회로로 확장 여지가 있다.
- 스캔 그룹 프리징/전환(scan_groups.py의 GSS 방식)은 이번 RTL 구현 범위에 포함하지 않았다.
- 정밀 타이밍 분석(합성 후 최대 동작 주파수 등)은 진행하지 않았으며, 필요 시 별도 EDA 툴체인 환경에서 추가 가능하다.

5. 핵심 버그 사례 심층 분석

검증 과정에서 발견된 결함은 성격이 다른 두 가지로 나뉜다. 하나는 테스트 인프라(테스트벤치) 자체의 문제였고, 다른 하나는 실제 RTL 설계 로직에 존재하던 결함이었다. 아래에 각각을 구분하여 문제-원인-해결-재검증 순으로 정리한다.

5.1 (경미) 테스트벤치 레이스 컨디션

구분	내용
증상	PRPG 테스트벤치 300 사이클 전부 불일치 (got이 매번 expected의 다음 값과 일치하는 패턴)
원인	테스트벤치가 clock의 posedge 직후 곧바로 제어 신호(enable)를 변경 → DUT의 always 블록과 같은 시점에서 경쟁(race condition) 발생 → 의도한 것보다 1 사이클 더 진행된 상태가 관측됨
해결	자극 변경 시점을 posedge 대신 negedge로 이동. 4개 테스트벤치(prpg/misr/controller/top) 전부 동일하게 수정
재검증	수정 후 전 스위트 재실행, 전부 PASS
성격 구분	RTL 로직 자체의 결함이 아니라 테스트 코드(자극 인가 타이밍)의 실수

표 3. 테스트벤치 레이스 컨디션 요약

5.2 (핵심) PRPG-MISR 간 NBA 타이밍 버그 — 이 프로젝트에서 가장 크리티컬했던 결함

이 버그는 실제 RTL 설계 로직에 존재하던 결함이며, 우연한 조건에 의해 통과 판정 뒤에 완전히 숨어 있었다는 점에서 이번 프로젝트 전체에서 가장 중요한 발견이었다.

구분	내용
발견 경위	게이트 레벨 CUT을 실제로 연결하는 확장 작업(3.5절) 도중, lbist_top 통합 결과를 다시 들여다보면서 발견됨
증상	MISR이 '이번 클럭 엣지에 새로 생성된 PRPG 패턴'이 아니라 '한 사이클 전 패턴'을 참조하고 있었음 — signature가 실제로 있어야 할 값보다 한 박자 늦게 갱신됨
근본 원인	PRPG의 상태 레지스터와 MISR의 시그니처 레지스터가 같은 클럭 엣지에서 동시에 갱신되는 구조였는데, 조합적으로 파생된 신호(다른 레지스터가 그 엣지에서 만들어낸 값)는 Verilog의 논블로킹 대입(NBA) 스케줄링 규칙상 같은 엣지 내에서는 아직 반영되지 않은 값으로 취급된다. 이는 시뮬레이터의 버그가 아니라, 실제 동기 회로 설계에서도 동일하게 발생하는 근본적인 타이밍 제약이다.
은폐된 이유	기존 통합 테스트가 PRPG 패턴 폭(WIDTH)을 64(16의 배수)로 설정했는데, MISR의 phase-shifter 오프셋 공식이 16주기 구조를 가져 이 폭에서는 매 채널이 정확히 4번씩 XOR-fold되어 시그니처가 항상 0으로 상쇄되었다. 시그니처가 늘 0이었기 때문에 RTL과 Python 레퍼런스가 '우연히' 계속 일치해, 실제로 존재하던 이 버그가 다수의 통과(PASS) 판정 뒤에 완전히 가려져 있었다.
왜 심각한가	이런 종류의 결함은 Verilog RTL로 옮기지 않았다면 원천적으로 존재할 수 없는 카테고리다. Python 시뮬레이션은 명령을 순차적으로 실행하므로 '같은 시점에 여러 레지스터가 동시에 갱신되며 서로의 새 값을 볼 수 있는지'라는 개념 자체가 없다. 즉 RTL로 실제 구현하지 않았다면 영원히 발견할 수 없었던 버그였다.

해결 방법	prpg.v 모듈에 현재 상태(state)가 아닌 '다음 상태(next_state)'를 조합적으로 노출하는 next_pattern 포트를 신규 추가하여, 같은 클록 엣지 내에서 MISR이 최신 패턴을 즉시 소비할 수 있도록 수정했다. 아울러 최상위 모듈(lbist_top)의 관측용 pattern/response 신호도 레지스터로 변경해, 외부 관측 시점이 post-edge 샘플링과 어긋나지 않도록 정렬했다.
재검증 방법	① 우연한 은폐를 배제하기 위해 검증 폭을 16의 배수가 아닌 63으로 변경하고 골든 레퍼런스를 재생성 — 수정 후 의미 있는(0이 아닌) 최종 시그니처(0x7c3e)로 4개 스위트 전부 재통과 확인. ② 게이트 레벨 CUT 결함 주입 테스트(3.5절의 확장 작업)도 이 수정 사항을 반영해 재실행 — 150 사이클 전부 일치, fault_sim.py(Python 결함 시뮬레이터)와 결함 검출 시점(패턴 50) 및 최종 검출 여부까지 정확히 일치함을 확인했다.

표 4. PRPG-MISR NBA 타이밍 버그 — 발견부터 재검증까지

정리하면, 이 버그는 '우연히 통과된 결과를 의심하고 검증 조건 자체의 타당성까지 재점검한 것'이 발견의 출발점이었고, 수정 후에는 더 엄격한 조건(비퇴화 폭, 실제 게이트 레벨 CUT)에서 재검증까지 완료된 사례다. 단순히 코드를 생성하는 것을 넘어, AI가 생성한 결과와 그 검증 과정 자체의 신뢰성까지 함께 검토해야 한다는 점을 보여주는 핵심 사례다.

부록 A. 최적 컨트롤러 RTL 파라미터

T_INIT = 33 T_MAX = 1024 DELTA_T = 193 K = 3 W = 34 CALIB_PATTERNS = 72 (= floor(33 × 2.183035341991451)) ALPHA_FRAC_BITS = 24 ALPHA_FIXED = 14111141
(round(0.8410895754890528 × 2^24))

부록 B. T-period 전환 이력 (검증 시퀀스 기준)

패턴 인덱스	T (변경 전 → 변경 후)
91	33 → 226
94	226 → 419
97	419 → 612
100	612 → 805
103	805 → 998
106	998 → 1024 (T_max에서 clamp)